



LABFluid
Laboratório de Dinâmica
dos Fluidos Computacional

Samuel Kutz Paranhos

Universidade Federal do Paraná (UFPR)
Laboratório de Dinâmica dos Fluidos (LabFluid)

Junho de 2026

Métodos de Machine Learning para Aproximação do Sistema de Boussinesq

Sumário

- 1 Objetivos
- 2 O Sistema de Boussinesq
- 3 Machine Learning
- 4 Geração do Conjunto de Dados

- 5 Métodos de Machine Learning
- 6 Metodologia
- 7 Resultados e Discussão
- 8 Conclusões

Objetivos

As Equações Diferenciais Parciais (EDPs) são uma das principais linguagens com que modelamos a natureza. Raramente possuem solução fechada, e por isso dependemos de **métodos numéricos** — tipicamente discretizando o domínio em malhas e avançando no tempo.



Uma ponte natural

Com a ascensão dos métodos de *Machine Learning* (ML) e de todo um ecossistema de *software* construído em torno deles, surge uma extensão natural: aproximar soluções de EDPs **aproveitando esse ferramental de Machine Learning** (ML).

Neste trabalho, o objeto de estudo é o **sistema de Boussinesq**, um modelo clássico da hidrodinâmica para ondas longas, não lineares e dispersivas em águas rasas.

Objetivo geral

Estudar a **eficácia** de três métodos de Machine Learning — **PINN**, **FNO** e **PINO** — para aproximar a solução do sistema de Boussinesq, sempre medindo o erro contra um **solver pseudoespectral** de alta acurácia tomado como referência.

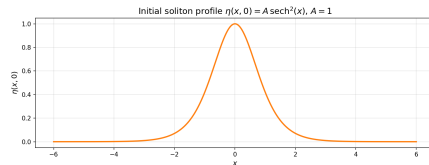
Três perguntas guiam o estudo:

- 1 **Acurácia vs. regime físico:** quão bem cada método reproduz a referência à medida que a não linearidade e a dispersão crescem? Onde cada um começa a falhar?
- 2 **Dados, física ou híbrido:** como a presença (ou ausência) de dados supervisionados afeta a acurácia e o treinamento? Adicionar um resíduo físico ajuda quando há poucos dados?
- 3 **Viés espectral:** como o *spectral bias* se manifesta em cada arquitetura, e em que medida dados ou física alteram *quais* frequências do erro são resolvidas primeiro?

O Sistema de Boussinesq

Em 1834, J. Scott Russell observou num canal escocês uma **onda solitária**: uma elevação de água que viajava por quilômetros *sem mudar de forma*.

Em 1872, J. Boussinesq explicou o fenômeno como um **equilíbrio** entre **não linearidade** e **dispersão**, dando origem ao modelo de ondas longas em águas rasas que estudamos aqui (Boussinesq 1872).



Condição inicial $\eta(x, 0) = A \operatorname{sech}^2(x)$.

Referências

Teoria em Whitham (1974) e Debnath (2012); forma moderna em Peregrine (1967) e Bona et al. (2002); parametrização de Martins (2023) e Martins et al. (2024).

Para a elevação da superfície $\eta(x, t)$ e a velocidade horizontal média $u(x, t)$, na forma unidimensional e periódica:

Sistema de Boussinesq

$$\begin{cases} \eta_t + u_x + \alpha (\eta u)_x = 0, \\ u_t - \frac{\beta}{3} u_{xxt} + \eta_x + \alpha u u_x = 0, \\ \eta(x, 0) = A \operatorname{sech}^2(x), \quad u(x, 0) = 0, \quad x \in [-L, L]. \end{cases}$$

- $\alpha \geq 0$: parâmetro de **não linearidade**;
- $\beta > 0$: parâmetro de **dispersão**.

No conjunto de dados deste trabalho fixamos $\alpha = \beta$: uma família de **um parâmetro** cuja dificuldade cresce com o valor comum.

Caso Limite: a Equação da Onda

Quando $\alpha = \beta = 0$, somem os termos não lineares e dispersivos, e o sistema reduz-se à equação da onda:

$$\eta_{tt} = \eta_{xx}.$$

Pela fórmula de d'Alembert, a elevação inicial se divide em dois pulsos que se afastam a velocidades ± 1 , sem mudar de forma:

Solução de referência ($\alpha = \beta = 0$)

$$\eta(x, t) = \frac{A}{2} \left[\operatorname{sech}^2(x - t) + \operatorname{sech}^2(x + t) \right].$$

Intuição

É exatamente esse comportamento — duas ondas se separando — que esperamos *recuperar* para $\alpha = \beta$ pequenos. Ao **aumentar** os parâmetros, surge o desvio: dispersão e não linearidade produzem perfis mais ricos, com energia espalhada por mais frequências, e *fundamentalmente mais difíceis de aprender*.

Machine Learning

Uma definição de trabalho

Em resumo, um método de Machine Learning pode ser visto como um **método numérico na presença de um conjunto de dados** e de um **objetivo**: dado um modelo paramétrico u_θ e uma *função de perda* $\mathcal{L}(\theta)$ que mede o quão longe estamos do objetivo, ajustamos os parâmetros θ por otimização (tipicamente descida do gradiente) até minimizá-la.

Os três ingredientes que reencontraremos em *todos* os métodos a seguir:

- 1 um **modelo** flexível e diferenciável (u_θ);
- 2 um **objetivo** codificado numa perda (dados, física, ou ambos);
- 3 um **otimizador** que segue o gradiente da perda.

- **1943** — neurônio formal de McCulloch e Pitts (1943);
- **1958** — *perceptron* de Rosenblatt (1958), o primeiro modelo treinável;
- **1986** — *backpropagation* (Rumelhart et al. 1986): torna prático o treino de redes profundas;
- **1989** — teorema da aproximação universal (Cybenko 1989; Hornik et al. 1989);
- **2015+** — *deep learning* (LeCun et al. 2015) e a revolução de IA em curso.

Um breve histórico

Por que agora?

Vivemos uma **expansão sem precedentes** da área: *hardware* (GPUs), *frameworks* de diferenciação automática (PyTorch) e uma vasta literatura tornaram esses métodos acessíveis.

Observação

Um dos objetivos deste trabalho é justamente **tirar proveito** desse ferramental moderno, trazendo-o para a solução numérica de EDPs.

Para resolver EDPs, três famílias guiam este trabalho:

PINN

Rede neural com a **física na perda** (Raissi et al. 2019). Resolve *uma* instância da EDP.

FNO

Aprende o **operador solução** entre espaços de funções, de forma puramente *data-driven* (Li et al. 2020).

PINO

Operador neural com **física na perda** (Li et al. 2023): o híbrido dos dois.

Lacuna que motiva o trabalho

Métodos de Machine Learning já foram aplicados a diversas equações de águas rasas e ondas dispersivas, mas, até onde sabemos, **nenhum estudo avaliou sistematicamente FNO, PINO e PINN no sistema de Boussinesq paramétrico 1D**, com não linearidade e dispersão variando simultaneamente.

Geração do Conjunto de Dados

Antes dos métodos: precisamos de dados

Todo método de ML precisa de um **conjunto de dados**. Aqui ele é gerado por um **solver pseudoespectral** de alta acurácia, que também servirá de *referência* para medir o erro:

$$\mathcal{S} = \{ ((\alpha_i, \beta_i), (\eta_i, u_i)) \}_{i=1}^M.$$

Ideia do método pseudoespectral

No domínio de Fourier, **derivar vira multiplicar**. Definindo o número de onda $\xi_k = \pi k/L$, para uma função periódica:

$$\widehat{(f_x)}(k) = i \xi_k \hat{f}(k), \quad \widehat{(f_{xx})}(k) = -\xi_k^2 \hat{f}(k).$$

Diferente das diferenças finitas, isto é *exato* (acurácia espectral).

Aplicando a DFT ao sistema de Boussinesq, cada modo k obedece a um **sistema de EDOs 2×2 no tempo**:

Sistema espectral (para cada $k \in \mathbb{Z}$)

$$\begin{cases} \hat{\eta}_t(k, t) = -i \xi_k \hat{u}(k, t) - \alpha i \xi_k \widehat{(\eta u)}(k, t), \\ \hat{u}_t(k, t) = \frac{-i \xi_k \hat{\eta}(k, t) - \alpha i \xi_k \widehat{(\frac{1}{2} u^2)}(k, t)}{1 + \frac{\beta \xi_k^2}{3}}. \end{cases}$$

- O operador dispersivo $1 - \frac{\beta}{3} \partial_{xx}$ vira a **divisão escalar** por $1 + \frac{\beta}{3} \xi_k^2$ — uma divisão por modo.
- Os termos não lineares $(\eta u, \frac{1}{2} u^2)$ são montados em **espaço físico** via FFT, a custo $O(N_x \log N_x)$.

Escrevendo o lado direito espectral como $F(\hat{\eta}, \hat{u}) = (\hat{\eta}_t, \hat{u}_t)$, a solução avança no tempo com o **RK4 clássico**:

Passo de RK4

$$\begin{aligned}\kappa_1 &= F(y_n), & \kappa_2 &= F\left(y_n + \frac{\Delta t}{2} \kappa_1\right), \\ \kappa_3 &= F\left(y_n + \frac{\Delta t}{2} \kappa_2\right), & \kappa_4 &= F(y_n + \Delta t \kappa_3), \\ y_{n+1} &= y_n + \frac{\Delta t}{6} (\kappa_1 + 2\kappa_2 + 2\kappa_3 + \kappa_4).\end{aligned}$$

A cada estágio, alterna-se entre **espaço físico** (produtos não lineares) e **espaço de Fourier** (derivadas) — a operação que define o método pseudoespectral.

Métodos de Machine Learning

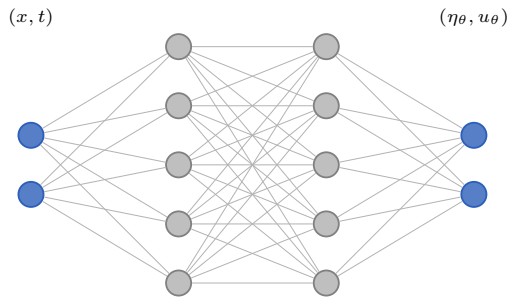
Bloco básico: redes neurais (MLP)

Uma *Multilayer Perceptron* (MLP) é uma composição de transformações afins com ativações não lineares σ (aqui, $\tanh$):

$$h^{(0)} = x, \quad h^{(l)} = \sigma\left(W^{(l)}h^{(l-1)} + b^{(l)}\right), \quad u_{\theta}(x) = W^{(L)}h^{(L-1)} + b^{(L)}.$$

Por aproximação universal (Cybenko 1989), uma MLP aproxima qualquer função contínua num compacto. Dado um conjunto $\{(x_i, u_i)\}$, ajustamos θ minimizando o erro quadrático médio:

$$\mathcal{L}_{\text{MLP}}(\theta) = \frac{1}{N} \sum_{i=1}^N \|u_{\theta}(x_i) - u_i\|^2.$$



Physics-Informed Neural Networks (PINNs)

Introduzidas por Raissi et al. (2017), as **PINNs** inserem a EDP *diretamente na perda*. Para um problema de valor inicial com operador diferencial $\mathcal{D}$:

$$\mathcal{D}[u](x, t) = 0 \text{ em } \Omega \times (0, T], \quad u(x, 0) = u_0(x).$$

Perda da PINN

$$\mathcal{L}_{\text{PINN}}(\theta) = \underbrace{\frac{1}{N_{\mathcal{D}}} \sum_i \|\mathcal{D}[u_{\theta}](x_i, t_i)\|^2}_{\text{resíduo da EDP}} + \underbrace{\frac{1}{N_0} \sum_j \|u_{\theta}(x_j, 0) - u_0(x_j)\|^2}_{\text{condição inicial}}.$$

Os resíduos são avaliados por **diferenciação automática**; a otimização usa Adam / L-BFGS. Se houver dados de referência, basta acrescentar **mais um termo**:

$$\mathcal{L}_{\text{PINN}} \longrightarrow \mathcal{L}_{\text{pde}} + \mathcal{L}_{\text{ic}} + \underbrace{\lambda_{\text{data}} \frac{1}{N_d} \sum_l \|u_{\theta}(x_l, t_l) - u_l\|^2}_{\mathcal{L}_{\text{data}}}.$$

Princípio da frequência (Rahaman et al. 2019; Xu et al. 2020)

Redes treinadas por gradiente reduzem as componentes de **baixa frequência** do erro **mais rápido** do que as de alta frequência.

Na prática, ao aproximar a solução:

- as componentes **suaves** (baixa frequência) emergem **primeiro**;
- os **detalhes finos** (alta frequência) demoram a aparecer — e, em regimes difíceis, podem **nunca** ser bem resolvidos.

Por que nos importa

À medida que $\alpha = \beta$ cresce, a solução de Boussinesq espalha energia por **mais frequências**. O viés espectral é um obstáculo **bem documentado** no treino de PINNs (Wang et al. 2022), e é justamente o que investigamos em cada arquitetura.

PINN para o sistema de Boussinesq

Treina-se uma PINN por amostra (α_i, β_i) . O operador $\mathcal{D}$ vira o par de resíduos de Boussinesq:

Resíduos

$$\mathcal{D}_1[u_\theta] = (\eta_\theta)_t + (u_\theta)_x + \alpha_i (\eta_\theta u_\theta)_x,$$

$$\mathcal{D}_2[u_\theta] = (u_\theta)_t - \frac{\beta_i}{3} (u_\theta)_{xxt} + (\eta_\theta)_x + \alpha_i u_\theta (u_\theta)_x.$$

$$\mathcal{L}_{\text{PINN}}^i(\theta) = \frac{1}{N_{\mathcal{D}}} \sum_l [\mathcal{D}_1^2 + \mathcal{D}_2^2] + \frac{1}{N_0} \sum_l [(\eta_\theta(x_l, 0) - \eta_0)^2 + (u_\theta(x_l, 0) - u_0)^2].$$

Como os parâmetros entram explicitamente nos resíduos, cada PINN codifica uma única amostra e precisa ser retreinada para cada novo par (α, β) .

O problema

É **muito difícil** construir uma PINN que funcione para *vários* parâmetros de uma EDP de uma só vez — cada parâmetro exige um novo treino.

Operadores neurais contornam isso: em vez de uma instância, aprendem o **operador solução** $G^\dagger : \mathcal{A} \rightarrow \mathcal{U}$ entre espaços de funções de dimensão infinita. Compõem operadores integrais lineares com não linearidades pontuais:

Definição geral de operador neural

$$G_\theta = Q \circ \sigma(W_L + \mathcal{K}_L) \circ \cdots \circ \sigma(W_1 + \mathcal{K}_1) \circ P, \quad (\mathcal{K}v)(x) = \int_D \kappa(x, y) v(y) dy.$$

P : *lifting* (eleva a dimensão de canais); Q : projeção de volta; $\mathcal{K}_\ell$: operador integral aprendido. No treino, basta uma **passada direta** para cada novo parâmetro.

O **FNO** (Li et al. 2020) escolhe um **kernel invariante por translação**, $\kappa_\ell(x, y) = \kappa_\ell(x - y)$. O operador integral vira *convolução*, eficiente no domínio da frequência pelo teorema da convolução:

Camada espectral

$$(\mathcal{K}_\ell v_\ell)(x) = \int_D \kappa_\ell(x - y) v_\ell(y) dy = \mathcal{F}^{-1}(\mathcal{F}(\kappa_\ell) \cdot \mathcal{F}(v_\ell))(x).$$

- › Em vez de aprender κ_ℓ , aprendem-se diretamente seus **coeficientes de Fourier** $R_{\ell,k} \approx 2\pi \hat{\kappa}_\ell(k)$ (pesos complexos);
- › retêm-se apenas os $|k| \leq k_{\max}$ modos de baixa frequência (hiperparâmetro);
- › resulta em **independência de malha** (*zero-shot super-resolution*).

Um cuidado no tempo: o fenômeno de Gibbs

Aplicar a transformada de Fourier **também no eixo do tempo** supõe que a solução é *periódica em t* — como se houvesse um *tiling* temporal.

Problema

A solução de Boussinesq **não** é periódica no tempo. A extensão periódica implícita cria uma *descontinuidade* na fronteira temporal, disparando o **fenômeno de Gibbs** (Gibbs 1899): oscilações artificiais que persistem por mais modos que se retenha.

Solução adotada (seguindo Li et al. (2020)): manter as camadas de Fourier apenas no *espaço* e tratar o tempo como **direção de marcha**. O FNO é treinado de forma puramente supervisionada:

$$\mathcal{L}_{\text{FNO}}(\theta) = \frac{1}{M} \sum_{i=1}^M \frac{1}{N_x N_t} \sum_{j,n} [(\eta_{\theta}^i - \eta_i)^2 + (u_{\theta}^i - u_i)^2].$$

O **PINO** (Li et al. 2023) faz com o FNO o que a PINN fez com a MLP: acrescenta o **resíduo físico à perda**. O *backbone* (FNO) é o mesmo; muda a perda.

Como o FNO produz (η_θ, u_θ) como um **array** $N_x \times N_t$, avaliam-se os resíduos por diferenciação numérica:

Perda do PINO

- derivadas **espaciais**: pseudoespectrais (via ξ_k);
- derivadas **temporais**: diferenças finitas.

$$\mathcal{L}_{\text{PINO}} = \lambda_{\text{pde}} \mathcal{L}_{\text{pde}} + \lambda_{\text{ic}} \mathcal{L}_{\text{ic}} + \lambda_{\text{data}} \mathcal{L}_{\text{data}}.$$

Por que importa

Com o resíduo físico, o PINO pode ser treinado com **menos dados rotulados**. Ele será estudado em duas variantes: **com dados** e **sem dados**.

Metodologia

- › **Linguagem/framework:** Python + **PyTorch** (diferenciação automática, GPU);
- › **Operadores neurais** (FNO/PINO): adaptados da biblioteca *neuraloperator* — camada **SpectralConv2d** com FFT e truncamento de modos;
- › **Solver de referência:** implementação pseudoespectral própria (FFT + RK4) que gera o *dataset* e serve de *baseline*;
- › **PINN:** MLP com tanh, resíduos por diferenciação automática.

Hardware modesto

Todos os experimentos rodaram numa única GPU **NVIDIA GTX 1660 SUPER** — uma placa de entrada. Um objetivo prático é mostrar que esses métodos são viáveis **sem infraestrutura de ponta**.

Observação

Reprodutibilidade garantida por *seed* fixa (37) e modo determinístico do cuDNN.

Os experimentos são desenhados para responder às três perguntas dos objetivos:

Variantes (dados vs. física)

- › **FNO**: puramente *data-driven*;
- › **PINO**: com dados *e* sem dados;
- › **PINN**: com dados *e* sem dados.

Regimes (acurácia vs. dificuldade)

Família $\alpha = \beta$ com 20 amostras em $[0,1, 4,0]$; avaliação em $\alpha = \beta \in \{0,1, 3,21, 4,2\}$ (inclui *extrapolação*).

Resolução (independência de malha)

Treino em $N_x = 128$; avaliação também em 256 e 512 para testar *super-resolution*.

Métricas

- › erro relativo L^2 no espaço-tempo;
- › erro relativo no **espectro de Fourier** (diagnóstico do viés espectral);
- › evolução do erro por frequência ao longo do treino.

Configurações e tamanho dos modelos

	FNO / PINO	PINN
Épocas	15,000	15,000
Otimizador	Adam	Adam
<i>Learning rate</i>	10^{-3}	10^{-3}
Modos (k_x, k_t)	16×16	—
Largura/neur.	32	256
Camadas	4	4
Pontos/ <i>batch</i>	256	5000+500

Tamanho dos modelos

- › **PINN**: $\approx 300\text{K}$ parâmetros;
- › **FNO / PINO**: $\approx 1\text{M}$ parâmetros.

Pesos da perda (PINO)

$\lambda_{\text{pde}} = 100$, $\lambda_{\text{ic}} = 1$, $\lambda_{\text{data}} = 1$.

Domínio: $x \in [-60, 60]$, $t \in [0, 30]$.

Resultados e Discussão

Os painéis (solução vs. referência, erro relativo e espectro de Fourier) e as animações de cada método são apresentados **ao vivo**. A seguir, os principais comentários por arquitetura.

O que observar

Em todos os casos, o comportamento de interesse aparece ao **aumentar** $\alpha = \beta$: é quando a solução espalha energia por mais frequências e o **viés espectral** e o **efeito de Gibbs** se tornam visíveis.

- › Inferência **muito rápida** — uma única passada direta, mesmo com $\approx 1\text{M}$ de parâmetros;
- › boa fidelidade nos regimes vistos no treino;
- › porém surge o **efeito de Gibbs** na direção temporal: oscilações artificiais que não desaparecem aumentando o número de modos.

- **Com dados:** o resíduo físico **elimina o Gibbs** e a aproximação fica consistente com a referência;
- a **propagação da condição inicial** no tempo é o ponto mais delicado do treino;
- **Sem dados:** reaparece um **viés espectral forte** — o PINO puramente físico tem dificuldade com os detalhes de alta frequência.

- A PINN **sofre viés espectral**: aprende primeiro as componentes suaves e perde as altas frequências do perfil;
- acrescentar um termo de **dados** *atenua* (mas não elimina) o efeito;
- **sem dados**, a dificuldade é mais severa à medida que a não linearidade cresce.

Comparando as arquiteturas

Por aprenderem diretamente no espaço de Fourier, o **FNO** e o **PINO** com dados sofrem menos com o viés espectral; ele reaparece com força no **PINO** sem dados e nas **PINNs**.

Conclusões

Método	Gibbs (tempo)	Viés espectral	Custo de inferência
PINN (sem dados)	—	forte	por amostra
PINN (com dados)	—	atenuado	por amostra
FNO	presente	moderado	muito baixo
PINO (com dados)	ausente	moderado	muito baixo
PINO (sem dados)	ausente	forte	muito baixo

- › A **PINN** sofre viés espectral; *com dados* ele é menos severo.
- › O **FNO** apresenta o efeito de Gibbs no tempo, mas é **extremamente rápido** (1M de parâmetros).
- › No **PINO** o Gibbs *some*, porém a propagação da condição inicial complica; *sem dados*, o viés espectral volta a ser forte.

O que (não) propomos

Este trabalho **não** propõe uma solução para a patologia. O foco é **exibir** que técnicas de Machine Learning, aplicadas ao sistema de Boussinesq, **reproduzem o viés espectral** — de modo que um usuário interessado nessas tecnologias *deve levá-lo em consideração*.

- Aplicar Machine Learning para capturar **ondas dispersivas** ainda **não é consolidado** para essas arquiteturas;
- é necessária **alguma mitigação** do viés espectral;
- estas técnicas se mostram um **complemento** aos métodos numéricos clássicos — **não** um substituto.

Hipótese de trabalho

O ML aprende o caráter **não linear** com facilidade, mas tem grande **dificuldade com o caráter dispersivo** da solução.

-  Bona, J. L., M. Chen e J.-C. Saut (2002). “Boussinesq equations and other systems for small-amplitude long waves in nonlinear dispersive media: I. Derivation and linear theory”. Em: *Journal of Nonlinear Science* 12, pp. 283–318. DOI: [10.1007/s00332-002-0466-4](https://doi.org/10.1007/s00332-002-0466-4).
-  Boussinesq, J. (1872). “Théorie des ondes et des remous qui se propagent le long d’un canal rectangulaire horizontal”. Em: *Journal de Mathématiques Pures et Appliquées* 17, pp. 55–108.
-  Cybenko, G. (1989). “Approximation by superpositions of a sigmoidal function”. Em: *Mathematics of Control, Signals and Systems* 2.4, pp. 303–314.
-  Debnath, L. (2012). *Nonlinear Partial Differential Equations for Scientists and Engineers*. 3^a ed. New York: Birkhäuser.
-  Gibbs, J. Willard (1899). “Fourier’s Series”. Em: *Nature* 59.1539, p. 606. DOI: [10.1038/059606a0](https://doi.org/10.1038/059606a0).
-  Hornik, K., M. Stinchcombe e H. White (1989). “Multilayer feedforward networks are universal approximators”. Em: *Neural Networks* 2.5, pp. 359–366.
-  LeCun, Y., Y. Bengio e G. Hinton (2015). “Deep learning”. Em: *Nature* 521.7553, pp. 436–444. DOI: [10.1038/nature14539](https://doi.org/10.1038/nature14539).

-  Li, Z. et al. (2023). “Physics-Informed Neural Operator for Learning Partial Differential Equations”. Em: *ACM/IMS Transactions on Data Science*. URL: <https://arxiv.org/abs/2111.03794>.
-  Li, Z. et al. (2020). “Fourier neural operator for parametric partial differential equations”. Em: *arXiv preprint arXiv:2010.08895*.
-  Martins, L. G. (2023). “Estudo Numérico do Modelo Boussinesq com Topografia e Pressão Variável no Espaço e no Tempo”. Diss. de mestr. Curitiba: Universidade Federal do Paraná.
-  Martins, L. G., M. V. Flamarion e R. Ribeiro-Jr (2024). “A forced Boussinesq model with a sponge layer”. Em: *Partial Differential Equations in Applied Mathematics* 10, p. 100661. DOI: 10.1016/j.padiff.2024.100661.
-  McCulloch, W. S. e W. Pitts (1943). “A logical calculus of the ideas immanent in nervous activity”. Em: *Bulletin of Mathematical Biophysics* 5.4, pp. 115–133.
-  Peregrine, D. H. (1967). “Long waves on a beach”. Em: *Journal of Fluid Mechanics* 27.4, pp. 815–827. DOI: 10.1017/S0022112067002605.
-  Rahaman, N. et al. (2019). “On the spectral bias of neural networks”. Em: *International Conference on Machine Learning (ICML)*. PMLR. Long Beach, pp. 5301–5310.

-  Raissi, M., P. Perdikaris e G. E. Karniadakis (2017). “Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations”. Em: *arXiv preprint arXiv:1711.10561*.
-  — (2019). “Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations”. Em: *Journal of Computational Physics* 378, pp. 686–707.
-  Rosenblatt, F. (1958). “The perceptron: a probabilistic model for information storage and organization in the brain”. Em: *Psychological Review* 65.6, pp. 386–408.
-  Rumelhart, D. E., G. E. Hinton e R. J. Williams (1986). “Learning representations by back-propagating errors”. Em: *Nature* 323.6088, pp. 533–536.
-  Wang, S., X. Yu e P. Perdikaris (2022). “When and Why PINNs Fail to Train: A Neural Tangent Kernel Perspective”. Em: *Journal of Computational Physics* 449, p. 110768. DOI: 10.1016/j.jcp.2021.110768.
-  Whitham, G. B. (1974). *Linear and Nonlinear Waves*. New York: Wiley-Interscience.
-  Xu, Z.-Q. J. et al. (2020). “Frequency Principle: Fourier Analysis Sheds Light on Deep Neural Networks”. Em: *Communications in Computational Physics*. Vol. 28. 5, pp. 1746–1767. DOI: 10.4208/cicp.OA-2020-0085.



Obrigado!